Building a Multi-Layer Knowledge Graph for Waste Disposal Guidance



Study program

Wirtschaftsinformatik Master WiSe 2025/2026

Course

Data Mining

Professor

Prof. Dr. Josef Fink

Submitted by

Riel, Niklas (1253801)
Hashemi, Arvand (1360176)
Rao, Niraj (1358243)
Cefariello, Luke (1350053)

Table of Contents

List of Abbreviations **3**
Executive Summary **4**
Introduction **5**
Initial Situation 5
The Case for Graph-Based Retrieval 6
Project Scope 7
Related Work **7**
Graph-based retrieval and hybrid search 7
Failure mitigation through verification and reflective loops 8
Knowledge graph construction and schema-first extraction 8
Retrieval pipeline trade-offs and context expansion 9
Multimodal inputs and modality gaps 9
Initial Evaluation of the Knowledge Base **10**
Web Search as a Drop-in Replacement 10
Unconstrained Reasoning and the Obsolescence of Guardrails 11
The Search for New Sources **12**
From Schema to Search Strategy 13
The Six Core Domains 14
Food and Grocery Products 14
Consumer Electronics and E-Waste 14
Chemicals and Hazardous Materials 15
German Municipal Waste Rules 15
AVV/EWC Legal Codes 16
Construction Materials 16
Additional Research Domains 17
Cross-Cutting Discoveries 18
The SaaS Oligopoly 18
Conditional logic as a universal pattern 18
Regional variation is real but bounded 18
The scale mismatch 18
From 100 Million Entities to a Practical Graph 18
Priority Matrix and Data Gaps 19
Implications for the Data Strategy 21
ETL and LLM Pipelines **21**
Structured ETL 21
Schema-Driven LLM Extraction 22
Two Modes: Chain and Agent 23
Provenance and Idempotency 23
Models and Cost 24
Extending the Pipeline 24

Data Imports **25**
Dictionary Baseline 25
Open Food Facts 25
Office Items and the BMEcat Gap 26
AVV Code Hierarchy 27
Import Summary 28
Results **28**
Data Mining Perspective 29
LLM extraction pipeline 30
AVV hierarchical classification 30
Classification quality versus input quality 30
Gaps & Limitations 30
Future Work **31**
Immediate Next Steps 31
Completing the AVV classification 31
Municipal vocabulary expansion 31
Data Source Integration 31
Municipal SaaS APIs 32
Chemical substance databases 32
LLM extraction from unstructured documents 32
Institutional Partnerships 32
University procurement catalogs 32
University-specific infrastructure data 33
Connecting Products to Disposal Knowledge 33
Graph Quality and Maintenance 34
Material taxonomy extension 34
Classification refinement 34
References **34**

List of Abbreviations

Abbreviation	Meaning
API	**A**pplication **P**rogramming **I**nterface
ASR	**A**utomatic **S**peech **R**ecognition
AVV	**A**bfall**v**erzeichnis-**V**erordnung
CPU	**C**entral **P**rocessing **U**nit
CRAG	**C**orrective **R**etrieval-**A**ugmented **G**eneration
CSV	**C**omma-**S**eparated **V**alues
EAN	**E**uropean **A**rticle **N**umber
ETL	**E**xtract, **T**ransform, **L**oad
EWC	**E**uropean **W**aste **C**atalogue
GPT	**G**enerative **P**re-**T**rained **T**ransformer
GPU	**G**raphics **P**rocessing **U**nit
GraphRAG	**Graph** + **R**etrieval-**A**ugmented **G**eneration
GTIN	**G**lobal **T**rade **I**tem **N**umber
JSON	**J**ava**S**cript **O**bject **N**otation
JSONL	**J**ava**S**cript **O**bject **N**otation **L**ines
LLM	**L**arge **L**anguage **M**odel
PWA	**P**rogressive **W**eb **A**pp
RAG	**R**etrieval-**A**ugmented **G**eneration
SaaS	**S**oftware **a**s **a** **S**ervice
SHA-256	**S**ecure **H**ash **A**lgorithm (256-bit)
SKOS	**S**imple **K**nowledge **O**rganization **S**ystem
SPARQL	**S**PARQL **P**rotocol **A**nd **R**DF **Q**uery **L**anguage

SQL	**S**tructured **Q**uery **L**anguage
SSE	**S**erver-**S**ent **E**vents
UID	**U**nique **Id**entifier
UPC	**U**niversal **P**roduct **C**ode
W3C	**W**orld **W**ide **W**eb **C**onsortium
XML	e**X**tensible **M**arkup **L**anguage

Executive Summary

This semester's work addressed a critical limitation of the Fessi waste disposal chatbot: a knowledge base containing only 548 waste items from a single source, paired with a vector-based retrieval system that could not handle the conditional logic inherent in waste disposal guidance. We replaced this foundation with a multi-layer knowledge graph served by Neo4j, populated through nine import pipelines built on two complementary architectures: deterministic structured ETL for sources with known schemas, and schema-driven LLM extraction for unstructured documents. Systematic research across ~20 topic areas identified roughly 400 viable data sources spanning chemical databases, product registries, municipal waste platforms, and legal code hierarchies. From these, we imported a representative subset: a German dictionary baseline of 329,231 concept clusters, of which 67,912 were promoted to waste items through LLM classification; 4.29 million food products with 555,227 packaging components from Open Food Facts; 56,000 product nodes from EU Ecolabel and Open Beauty Facts to partially close the office-items data gap; and the complete AVV legal waste code hierarchy with a hierarchical three-step classification pipeline achieving a 99.8% success rate. The resulting graph contains over 5 million nodes and approximately 6 million edges, organized into five layers: vocabulary, core waste entities, products, disposal logic, and infrastructure. Three cross-cutting findings shaped the architecture. First, conditional disposal logic, where the same item routes differently depending on its state, appeared independently in every domain we researched, validating the schema's condition-based branching model. Second, three SaaS platforms consolidate most German municipal waste management, reducing the integration surface from hundreds of custom scrapers to three API endpoints. Third, regional disposal variation clusters around a small number of genuinely ambiguous items rather than being arbitrary across municipalities. The pipelines, schema, and prioritized source catalog provide a foundation that subsequent project teams can extend without rebuilding the extraction infrastructure.

Introduction

The effective management of waste disposal on university campuses presents a deceptively complex challenge. What appears straightforward ("Where do I throw this away?") quickly reveals layers of conditional logic, regional regulations, and situational context that no simple lookup table can address. A paint can, for instance, may require fundamentally different disposal routes depending on whether it still contains liquid paint or is empty and dry, a distinction that determines whether it constitutes hazardous waste or ordinary recyclable packaging. This kind of conditional reasoning, common across hundreds of waste categories, is precisely where traditional information retrieval systems fail.

"Fessi" is an intelligent waste disposal chatbot developed for the Frankfurt University of Applied Sciences campus. The project, now in its third semester of iterative development, aims to provide students, staff, and visitors with precise, context-aware disposal guidance through a conversational interface. The first semester produced "Müll-Meister," an initial prototype built on jQuery and Microsoft frameworks that demonstrated the concept but suffered from fundamental architectural limitations, including initialization times exceeding 58 seconds and a monolithic backend structure. The second semester saw a complete rewrite: our team rebuilt the frontend as a Progressive Web App using Angular, developed the API specifications, and delivered a fully functional chat interface with voice input, image upload, and offline capabilities. Concurrently, the backend was migrated to a LangGraph-based agent architecture, but the underlying retrieval mechanism remained a rigid, vector-based RAG pipeline that was not designed to handle the conditional logic inherent in waste disposal queries. As a result, the chatbot's performance was severely hampered by long response times and high token usage due to the need to retrieve and process large amounts of often irrelevant information, rendering it effectively unusable.

Initial Situation

Upon beginning this semester's work, we inherited a system that was architecturally sound but operationally limited. The previous backend/pipeline team had migrated the chatbot from a fixed processing pipeline to a LangGraph-based agent (a significant step forward). However, the agent still operated under strict guardrails inherited from the original pipeline design: a rigid sequence of vector search, follow-up question evaluation, followed by yet another vector search before starting the final response formulation. Since the scope and in particular the long inference times had not been shared with the other groups, they were not included into the design specifications, resulting in response times of up to 30 seconds during which the user interface provided no feedback whatsoever. Something which hurt the usability to a point that it rendered the chatbot effectively "unusable". More critically, upon examining the underlying knowledge base, we found remarkably little to build upon. After sorting/importing everything available, the system contained approximately 548 waste items sourced almost entirely from the Abfall-ABC. This limited coverage meant that the majority of user queries simply could not be answered from the knowledge base. In practice, the agent would search the graph or vector

store, find nothing relevant, and answer from its parametric knowledge which often led to generic responses who weren't grounded in the actual data.

The Case for Graph-Based Retrieval

The existing system relied on vector-based Retrieval-Augmented Generation (RAG), which retrieves document chunks by semantic similarity and passes them to the language model for response generation. This approach works well for straightforward factual lookups ("What are the opening hours of Wertstoffhof Nord?") where the answer exists verbatim in a document. However, it fundamentally fails on conditional disposal logic, which accounts for a significant portion of real waste disposal queries.

Consider the "Paint Can" problem: a user asks "How do I dispose of a paint can?" A vector search retrieves all document chunks mentioning paint cans, which simultaneously contain conflicting information: "dried paint residue goes in residual waste," "liquid paint must be taken to a hazardous waste facility," "empty metal cans belong in the packaging bin." The language model receives these contradictory passages and must somehow synthesize a coherent answer, typically resulting in a vague response that lists all options without determining which applies.

A knowledge graph models this problem as a decision tree rather than a collection of documents. The waste item "paint can" connects to a disposal rule, which branches on a condition ("Is it still liquid?"). The true branch routes to the hazardous waste facility; the false branch routes to the packaging bin. When the chatbot agent traverses this graph, it encounters the condition node and can ask the user a clarifying question before providing a precise, unambiguous answer. This is not a theoretical advantage; it represents the difference between "here are some things you could do with a paint can" and "bring it to the Schadstoffannahme on Campus Nibelungenplatz, open Tuesday and Thursday 10–14 Uhr."

Beyond conditional logic, graph-based retrieval offers structural advantages for an agentic chatbot. Rather than retrieving a fixed number of similar text chunks, the agent can traverse relationships, following links from a waste item to its disposal rules, from rules to facilities, from facilities to opening hours and addresses. This traversal mirrors the way a knowledgeable human would reason about waste disposal: identify the item, determine its category, check for special conditions, and locate the appropriate disposal point. A knowledge graph makes this reasoning path explicit and queryable.

Project Scope

This semester's work focuses on building the knowledge graph infrastructure that the chatbot needs to provide accurate, context-aware disposal guidance. This encompasses two complementary workstreams: constructing ETL pipelines that transform raw data sources into a structured Neo4j knowledge graph, and developing LLM-based extraction methods that can process unstructured German documents into graph-compatible entities and relationships.

The project also addresses the data scarcity problem identified in the initial situation. Through systematic research, we identified roughly 400 viable data sources for waste disposal knowledge, ranging from chemical substance databases with millions of entries to municipal PDF guides and legal statutes. Importing and classifying this data at scale, using both deterministic pipelines for structured sources and LLM-based extraction for unstructured documents, forms the core technical contribution of this work.

In parallel, we made targeted improvements to the chatbot agent itself. We replaced the rigid pipeline with a freely reasoning agent, removing the guardrails that had previously constrained it to a fixed sequence of tool calls. We implemented visible reasoning steps (inspired by the "thinking" displays in modern AI interfaces) so that the user could follow the agent's progress rather than staring at a blank screen. We added web search as a fallback knowledge source, and built a barcode scanning module that can decompose real-world products into their individual packaging components for per-material disposal routing, built directly into the knowledge graph.

This documentation is split across two companion documents reflecting the two modules under which this work was conducted. This document covers the **Data Mining** aspects: LLM-based information extraction, schema-driven entity extraction, document analysis and chunking, the LangGraph orchestration pipeline, LLM classification for the dictionary baseline, and barcode-based product matching.

Related Work

Recent practitioner-oriented work on Retrieval-Augmented Generation (RAG) emphasizes that production quality depends less on “better embeddings + larger top-k” and more on workflow and architecture decisions that make systems reliable, testable, and debuggable. A recurring point is that retrieval should be treated as only one part of the pipeline, and that verification and controlled generation are equally important, especially in domains where incorrect guidance has real consequences (Tomar, 2026; Yadav, 2026).

Graph-based retrieval and hybrid search

Classic vector-based RAG retrieves text chunks by similarity and then generates an answer from the retrieved context. This works well for direct fact lookups, but it tends to break down when answers require combining multiple constraints or following rule-like dependencies across passages. GraphRAG-style approaches address this by connecting evidence through entities and relations and enabling multi-hop reasoning with traceable provenance, which is particularly relevant for rule-driven domains (Tomar, 2026).

In practice, several sources frame GraphRAG less as a replacement and more as a complement to vector retrieval. Curry describes hybrid setups that combine semantic retrieval

with structured graph queries, for example by using vector search to find relevant candidates and then applying graph traversal or Cypher queries to enforce “hard” relations and constraints. This hybrid view reduces the risk that similarity alone determines the final answer when explicit structure is needed (Curry, 2026).

Failure mitigation through verification and reflective loops

A central theme is that strong retrieval alone does not guarantee correctness. Tomar argues for a “contrarian move”: investing as much engineering effort into verification and conservative generation as into retrieval itself. As a concrete pattern, Corrective RAG (CRAG) is presented as a loop that generates an initial answer, extracts claims, retrieves targeted evidence, and then verifies and rewrites; Tomar reports a case where the hallucination rate dropped by 78% after introducing CRAG. The same source also proposes selection heuristics, such as using HyDE or Self-RAG when documentation is sparse or phrasing varies strongly, because these patterns can improve recall via query reformulation or model-driven query expansion (with guardrails) (Tomar, 2026).

From an orchestration perspective, Yadav frames scalability primarily as a workflow-design problem and summarizes five production patterns (RAG, prompt chaining, routing, orchestrator-workers, and evaluator-optimizer) rather than focusing on a single model choice. Routing is positioned as a cost optimization by sending simple requests to cheaper handlers, while orchestrator-workers and evaluator-optimizers enable dynamic decomposition and iterative refinement but require clear stopping rules and careful context management to avoid token waste and debugging issues. Across patterns, Yadav emphasizes instrumentation (latency, token usage, errors) and caching; semantic caching is described as reducing API calls by up to 70% in some applications (Yadav, 2026). Survey work on tool learning frames orchestration as recurring stages such as planning, tool selection, execution, and response generation, and discusses the same robustness vs latency vs debuggability trade-offs in a more systematic way (Chen et al., 2025).

Knowledge graph construction and schema-first extraction

For knowledge graph construction, Curry describes an end-to-end pipeline view: define a schema, extract entities and relations, resolve entities consistently, and then store/query the graph for reasoning and traceability. A key recommendation is to start with a simple schema and iterate, because the schema determines what can be validated and queried later. Curry also highlights that stable identifiers and entity resolution (e.g., linking to external knowledge bases) matter for reliability, since inconsistent entities undermine traversal quality and provenance (Curry, 2026).

Downstream mining steps also depend on whether extracted outputs reliably conform to a predefined schema, since later logic typically requires machine readable fields rather than free text. SLOT proposes a model agnostic post processing layer that converts unstructured LLM

outputs into JSON matching a target schema and evaluates schema accuracy alongside content fidelity (Wang et al., 2025).

Beyond answer provenance in retrieval, mining pipelines often require artifact level provenance, for example stable identifiers for extracted objects, explicit derived from links back to source artifacts, and idempotent re ingestion so repeated runs do not create duplicates (W3C, 2013).

Retrieval pipeline trade-offs and context expansion

Silfverskiöld provides a useful counterpoint by showing how RAG pipelines can become “over-engineered” through stacked optimizations (multi-query generation, hybrid search, reranking, and context expansion). The source stresses that each added layer can increase latency and operational complexity, which makes failures harder to diagnose. A practical mitigation discussed is to keep chunks small for retrieval but expand context after a hit by pulling neighboring chunks (“expansion”) to preserve coherence. The same source also notes that structured inputs such as spreadsheets require explicit handling (e.g., validating key columns), because automatic extraction can otherwise introduce subtle retrieval and joining errors (Silfverskiöld, 2025).

Multimodal inputs and modality gaps

Although many RAG examples focus on text, practical systems often need to handle mixed inputs (documents, tables, and potentially images). Curry treats multimedia sources as part of KG pipelines (e.g., using OCR for images) and frames this as a way to bring non-text evidence into a structured form that can be queried consistently. Complementing this, Bansal discusses “multimodal GraphRAG” and motivates going beyond pure vector similarity when retrieval must connect information across modalities, which is relevant for future extensions where users provide photos in addition to text (Bansal, 2025; Curry, 2026).

Beyond retrieval, downstream classification can involve very large label spaces where scoring all labels is impractical. Recent work on extreme multi label classification emphasizes scalable label representations and efficient candidate selection, which provides a relevant reference point for hierarchical catalog style classification tasks (Ortego et al., 2026).

Product identification and barcode based linking can be framed as entity matching under real world heterogeneity, where schema differences, naming variation, and data quality drive many errors and therefore need to be handled explicitly in operational pipelines (Moslemi et al., 2026).

Initial Evaluation of the Knowledge Base

Our original plan for this semester had begun relatively straightforward: take the existing data, migrate it from the vector store into a knowledge graph, and compare the two retrieval approaches empirically. We had planned to measure whether graph-based retrieval produced more accurate/contextual answers than vector similarity search, particularly for the conditional

disposal queries described in the previous section. The chatbot improvements documented in the companion Data Engineering document (visible reasoning, streaming, expanded toolset) were meant to help build the evaluation environment for this comparison.

The plan did not survive contact with the data however. After importing everything available (the 548 waste items from the Abfall-ABC), we built the initial knowledge graph and began testing queries. The results were sobering. For simple, well-covered items like "Altpapier" or "Biomüll," the agent found the correct disposal route on the first lookup. But these items represented only a fraction of what users might ask about. Queries about batteries, paint, chemicals, electronics, cooking oil, or any of the dozens of items that require conditional handling simply had no corresponding data in the graph. The agent would search, receive an empty result, and either admit it had no information or, worse, fall back on its parametric knowledge to produce a plausible-sounding but ungrounded response.

We could not evaluate graph-based retrieval against vector-based retrieval when neither system had enough data to answer most queries. Rather than abandon the evaluation, we looked for a way to test the agent's reasoning capabilities without being blocked by the data gap.

Web Search as a Drop-in Replacement

The solution was to give the agent access to a web search tool, not meant as a permanent replacement, but a proxy for the knowledge base we had not yet built. We integrated the Tavily search API alongside the existing Neo4j tools, with no special fallback logic or priority ordering. From the agent's perspective, web search was simply another tool in its toolbox: it could choose to search the graph, search the web, or do both, depending on what it deemed necessary.

The effect was immediate and striking. With web search available, the agent could answer every query we tested, not with generic summaries, but with specific, actionable detail. What made this significant for our project was not the quality of the individual answers (a web search will always have broader coverage than a domain-specific knowledge base) but the pattern of how the agent assembled them. The agent did not issue a single broad query and summarize the results. It conducted targeted, iterative investigations: starting with a general question, identifying gaps in the initial results, and issuing follow-up queries to fill them.

For example, when asked about how to dispose of a bottle of hydrochloric acid in the city of Bremen, the agent issued multiple successive web searches. The agent progressively narrowed his search, from regulatory frameworks to specific services to the exact locations, proving it was capable of performing the multi step problem solving required for working with a well-structured knowledge graph. Therefore we were confident that the agent had the ability to start at a waste item node, follow the edge to its disposal rule, check the condition, traverse to the appropriate facility, and read off the address and opening hours. A knowledge graph, populated with the right data and connected by the right relationships, would support exactly this traversal pattern, but with higher precision and without dependence on external search APIs.

This insight shifted our project strategy. Rather than waiting until the knowledge graph was fully populated before evaluating the agent, we could use web search as a development-time proxy: validating the agent's reasoning architecture, refining its tool-use prompts, and identifying the categories of information it would need, all before investing the effort of building comprehensive data pipelines. The web search tool remains in the deployed system as a fallback for queries that fall outside the knowledge graph's current coverage.

Unconstrained Reasoning and the Obsolescence of Guardrails

The iterative search behavior described above would not have been possible under the previous agent architecture. As described in the “Initial Situation”, the inherited system constrained the agent to a fixed four-step sequence regardless of query complexity. This pipeline existed because GPT-4o, the previous model, could not reliably orchestrate a multi-step tool-use strategy on its own. Replacing GPT-4o with a reasoning model and removing the fixed pipeline entirely (a change described in detail in the companion Data Engineering documentation) allowed the agent to decide for itself which tools to call, in what order, and when to stop.

What mattered for our evaluation was not the technical mechanism but the behavioral consequence. A straightforward lookup triggered a single graph query. A conditional question triggered a sequence of searches, condition checks, and facility lookups. When the knowledge base returned no results, which given the sparse data happened quite frequently, the agent pivoted to web search without any explicit fallback logic in the code. On complex queries, we observed the agent making dozens of tool calls in sequence, refining search terms based on partial results and cross-referencing facilities. The decision to try an alternative tool was made by the model's internal reasoning, not by a conditional branch in the codebase.

This graceful degradation was particularly valuable for our evaluation. Under the old system, if the second vector search returned nothing, the pipeline continued to the response formulation step, producing an answer from whatever partial information it had. Under the new system, the agent could recognize that its initial search had failed and try a different approach (a broader search term, a category-level query, or a web search) before composing its response. By watching how the agent navigated around gaps in the knowledge base, we could identify which categories of information were most frequently missing. These observations directly influenced the data sourcing strategy described in later sections.

The Search for New Sources

The knowledge graph schema, designed in parallel with this research and documented in full in the companion Data Engineering documentation, organizes the graph into five layers, each serving a distinct purpose.

Layer	Name	Key Node Types	Purpose

0	Vocabulary	Dictionary Entry	Multilingual discovery, synonym matching
1	Core Entities	WasteItem, WasteStream, AVVCode	Central disposal concepts and legal classification
2	Products	Product, PackagingComponent, Material	Per-component disposal routing for packaged products
3	Logic & Rules	DisposalRule, Condition, Instruction	Conditional disposal logic and handling instructions
4	Infrastructure	Facility, Container, Building, Room	Physical locations and provenance tracking

The central traversal pattern is consistent regardless of entry point.

1. Identify the item (Layers 0–2)
2. Resolve its disposal rule and evaluate any conditions (Layer 3)
3. Arrive at a waste stream, and locate the nearest collection point (Layer 4)

A Product decomposes into PackagingComponent nodes linked to Material nodes, each of which maps to a WasteStream, enabling per-component disposal routing from a single barcode scan. DisposalRule nodes branch on Condition nodes (see the "Paint Can" problem discussed previously), with IF_TRUE and IF_FALSE edges routing to different waste streams. Provenance tracking via DERIVED_FROM edges to Source nodes applies across all layers.

After importing the Abfall-ABC and facility data (548 waste items and 38 facilities), most of these layers were empty. The vocabulary layer had no entries. The product layer (which didn't even exist at this point) had no products. The rules layer had no conditional logic, and the infrastructure layer only had 38 facilities from the previous semester's dataset. This section describes our search for information in an attempt to fill the knowledge graph.

From Schema to Search Strategy

Rather than searching for waste disposal data in general, we used the schema layers as an organizing framework. Each layer implies a category.

- The vocabulary layer needs dictionaries and thesauri
- The core layer needs municipal waste directories and legal code hierarchies
- The product layer needs databases with barcodes, packaging breakdowns, and material taxonomies
- The rules layer needs regulatory documents and hazard classification systems
- The infrastructure layer needs facility databases and municipal collection APIs

We formalized this mapping into ~20 topics, organized into four priority tiers.

- Tier 1 targeted the highest-impact data categories
    - Food and grocery products
    - Consumer electronics
    - Chemicals
    - Hazardous materials
    - German municipal waste rules
    - AVV legal codes
- Tier 2 covered domain-specific categories
    - Construction materials
    - Packaging and recycling codes
    - Batteries
    - Wood and furniture
    - Facility infrastructure
- Tier 3 addressed enrichment data
    - Material properties
    - Universal product taxonomies
    - Multilingual synonyms
    - Textiles
    - Disassembly instructions
- Tier 4 covered advanced features
    - Computer vision datasets
    - Transport regulations
    - Medical waste
    - Office supplies (added later on)

For each topic, we defined structured research criteria specifying what data we needed, how it should map to the schema, and what formats and licenses were acceptable. Each topic was investigated systematically based on the following criteria.

- Relevance to the knowledge graph
- Data coverage and completeness
- Format and access method
  (free API vs. registration required, structured vs. unstructured data)
- Licensing constraints

The initial sweep identified several hundred potential sources; after “rough” deduplication, we ended up with ~400 sources across all topics.

The Six Core Domains

The following domains produced the most significant findings. Each is summarized below with representative examples.

Food and Grocery Products

Food items are among the most common waste disposal queries, but the disposal question is rarely about the food itself, since organic waste goes in the Biotonne. The real complexity lies in the packaging: a single product can involve glass, plastic, cardboard, and metal components, each routed to a different bin. Open Food Facts emerged as the standout source for this domain: over 4 million products globally, more than 800,000 with German market data, and, crucially, a structured packaging array that provides per-component breakdowns with material, shape, weight, and recycling hints. This is exactly the decomposition data our product layer was designed to consume. The entire dataset is available as a Parquet bulk dump, eliminating API rate limit concerns for large-scale import. We also evaluated Open Beauty Facts for cosmetics, Open Pet Food Facts for pet products, and GS1 GEPIR for barcode verification, though Open Food Facts accounts for the vast majority of the product layer.

Consumer Electronics and E-Waste

Electronics disposal in Germany is governed by the ElektroG (Elektro- und Elektronikgerätegesetz), which classifies devices into six collection groups based on physical dimensions and device type, ranging from heat exchangers and screens to small IT and telecom equipment, with a key threshold at 50 cm (any dimension above this classifies a device as "large equipment"). Stiftung EAR, the official German producer registry, publishes this taxonomy as structured CSV data and serves as the authoritative source for mapping devices to their legal collection group.

A recurring question in electronics disposal is whether a battery can be removed before the device is discarded. iFixit's teardown database serves as a practical proxy: if disassembly difficulty is rated "Low" or "Moderate," the chatbot can instruct the user to remove the battery; if "Very Hard," the entire device should go to an e-waste collection point. We also evaluated Icecat for product specifications, EPREL for EU energy labels, ECHA's SCIP database for hazardous substances in products, and the Open Repair Alliance for failure mode statistics. Electronics disposal logic maps well to our conditional rule model but requires combining multiple sources: EAR for legal categories, Icecat for physical specifications, and iFixit for disassembly guidance.

Chemicals and Hazardous Materials

This domain offered the largest volume of available data by far. PubChem alone contains 111 million compounds with GHS hazard classifications, synonyms in multiple languages, and physical properties such as flash point and boiling point. For a university with various laboratories (though mostly “engineering focused”), even a small subset of common solvents, acids, bases, and reagents requires reliable hazard data to determine whether a substance

qualifies as hazardous waste. GESTIS, the German occupational safety database maintained by the DGUV, emerged as the gold standard for the German lab context: its 8,800 substance entries include storage classes (Lagerklassen) critical for waste segregation, handling instructions, and references to TRGS regulations that govern laboratory waste classification.

A significant discovery in this domain was that the BAM dangerous goods database, which provides UN transport numbers, ADR classification codes, and packing group assignments for 4,500 substances, transitioned from a paid subscription service (previously costing several hundred euros per year) to free public access in July 2025. This could eliminate the need for expensive logistics software or manual ADR lookups. We also evaluated the ECHA Classification and Labelling Inventory (5 million classifications), CAS Common Chemistry, EPA CompTox databases, and tools for automated Safety Data Sheet parsing. The challenge in this domain is not finding data but filtering it: our graph needs perhaps 500 to 2,000 common lab chemicals, not the full 111 million compounds we discovered.

German Municipal Waste Rules

This domain produced the most surprising structural finding. German municipal waste management appears hopelessly fragmented, with over 400 districts, each with its own waste management company, its own Abfall-ABC, and its own sorting rules. Building custom scrapers or parsers for each district would be intractable. But the research revealed that three SaaS platforms consolidate most of this landscape: Abfall+ (abfall.io), which dominates in southern and western Germany; MyMüll (operated by JumoMind), which covers much of NRW, Hesse, and western districts; and Müllmax, a legacy system used by several large cities. Rather than 400 custom integrations, three API integrations could therefore be enough to cover the majority of German university towns (something that might come in handy should Fessi be shared with other organizations).

The discovery was amplified by a second finding: an open-source Home Assistant integration (hacs_waste_collection_schedule) has already reverse-engineered these APIs and provides pre-configured tenant keys and municipality IDs for hundreds of cities. This represents a ready-made bootstrap for any waste data integration project.

Additionally we also identified Berlin's BSR Open Data as a vocabulary gold standard: over 2,000 waste items with synonyms, avoidance tips, and recycling instructions, published under a CC BY license. Because the waste items themselves are universal (a pizza box exists in every city), BSR's vocabulary could serve as a base ontology.

AVV/EWC Legal Codes

The Abfallverzeichnis-Verordnung implements the European Waste Catalogue as German law: 20 chapters organized by waste origin, approximately 113 subgroups, and roughly 840 specific six-digit waste codes. Codes marked with an asterisk denote hazardous waste. The authoritative source is Gesetze-im-Internet, the official German federal legal website, which provides the complete AVV as HTML and PDF.

The most complex aspect of the AVV is its mirror entries: pairs of codes for the same waste, distinguished by whether hazardous substances exceed concentration thresholds.

- Wood containing hazardous substances is classified as 20 01 37*
- Wood without hazardous substances is 20 01 38
- Soil contaminated with dangerous substances is 17 05 03*
- Clean soil is 17 05 04

The graph must link these pairs and trigger clarifying questions, which is exactly what the Condition nodes in our schema were designed for. The mirror entry pattern appeared independently in every domain we researched: chemicals, construction materials, electronics, and municipal waste all contain items where the disposal route depends on a condition that must be evaluated (something which would be up to the AI model to pick up when relying purely on vector search).

A particularly valuable source for bridging user language and legal classification is the SEPA Waste Thesaurus, published by Scotland's environmental agency. It maps thousands of common-language terms directly to EWC codes ("acetone" to 14 06 03*, "pallets" to 15 01 03, "fluorescent tubes" to 20 01 21*), providing the synonym layer that connects how people talk about waste to how the law classifies it. We also evaluated EU Vocabularies in SKOS format (natively compatible with knowledge graphs), UK WM3 guidance for mirror entry threshold calculations, LAGA technical guidelines, and university-specific chemical waste guides from ETH Zürich and LMU Munich.

Construction Materials

Construction and demolition waste is generally the largest waste stream by mass, but the least well-served by open data. There is no "Open Rubble Facts" (e.g. a demolition equivalent for Open Food Facts), no single database that provides comprehensive material classifications, densities, and contamination indicators for construction materials the way Open Food Facts does for food products.

The critical data challenge in this domain is density. A cubic meter of solid concrete has a density of approximately 2,400 kg/m³. The same volume of crushed concrete rubble has a bulk density of roughly 1,500 kg/m³. Mixed construction waste (rubble combined with wood, plastics, and wallpaper) can be as low as 200 kg/m³. Using the wrong density factor causes massive errors in logistics planning and disposal cost estimation. The graph must distinguish solid density (Rohdichte) from bulk density (Schüttdichte) and select the correct factor based on the AVV code. For density data, we had to federate values from multiple regional sources: the Bavarian statistics office, the City of Vienna's disposal conversion factors, and weighbridge measurements from South Tyrolean recycling facilities.

ÖKOBAUDAT, the German government's building material LCA database, provides life-cycle assessment data for over 1,400 building products, including end-of-life scenarios. LAGA M20

defines pollutant concentration limits that determine whether construction waste can be freely reused (class Z0), reused with restrictions (Z1/Z2), or must be treated as hazardous waste. Both are well-structured sources, but neither alone is sufficient, since construction waste data requires combining regulatory classification, material science, and operational logistics sources.

Additional Research Domains

Beyond the core domains, the remaining research topics covered material-specific categories, enrichment sources, and advanced capabilities. For packaging and recycling codes, we documented the complete international numbering system (codes 01–84 covering plastics, paper, metals, glass, and composites) and identified Open Food Facts material tags as a practical implementation of this taxonomy. For batteries, we discovered chemistry types (lithium-ion, NiMH, lead-acid, NiCd), their UN transport numbers, and BattG compliance requirements. For wood and furniture, we have the German Altholzverordnung's contamination classification from A I (untreated, freely recyclable) through A IV (preservative-treated, hazardous waste), another domain where condition-based disposal routing is essential.

Enrichment sources included Wikidata, which provides material properties, recycling codes, and labels in over 300 languages through its SPARQL endpoint; UNSPSC, a product taxonomy with 157,000 codes available as a free PDF; and ECLASS, the German industry classification standard (licensed, not free).

For computer vision, we evaluated TACO (1,500 annotated images, 60 waste categories) and TrashNet (2,527 images, 6 categories), both small datasets useful for prototyping but insufficient for production deployment.

Laboratory and Medical waste (AVV Chapter 18, LAGA M18 guidelines) is particularly relevant for the university context but involves regulatory complexity that exceeds what we could address within the semester.

Office supplies proved to be the domain with the largest open-data gap. No equivalent to Open Food Facts exists for everyday stationery and office items. The EU Ecolabel database and Blauer Engel product registry cover eco-certified products but not the full range of items a university procures.

Cross-Cutting Discoveries

Four structural insights emerged across the domain-specific research and shaped our overall data strategy.

The SaaS Oligopoly

The discovery that three SaaS platforms consolidate most German municipal waste management has implications beyond the municipal rules domain. It means that facility data,

collection schedules, and even Abfall-ABC vocabularies can be sourced through a small number of API integrations rather than hundreds of municipal scrapers. Combined with the pre-configured keys available through the open-source Home Assistant integration, this transforms what appeared to be the most fragmented data landscape into one of the most accessible.

Conditional logic as a universal pattern

The AVV mirror entry pattern (same waste, two codes depending on contamination) is not a legal curiosity confined to the AVV chapter. It appeared independently in every domain we researched. Electronics may or may not contain hazardous components. Wood may or may not be treated with preservatives. Paint may or may not still be liquid. The pattern is the same: a disposal route that branches on a condition. This confirmed that the schema's Condition-based branching model was not overengineered for edge cases but reflects a fundamental structure in waste classification law.

Regional variation is real but bounded

The pizza box paradox and bioplastics consensus described previously illustrate that disposal rules vary by municipality, but the variations are not arbitrary. They cluster around a small number of genuinely ambiguous items. A graph with city-level scope overrides, as our schema supports through the DisposalRule's scope property, can model this without duplicating the entire rule set for every municipality.

The scale mismatch

The raw data landscape exceeds 100 million potential entities, but our university focused waste disposal chatbot does not need to know about every compound ever synthesized or every product ever manufactured. The challenge we faced was not data acquisition but distillation. The next section will describe how we reasoned about what to actually import.

From 100 Million Entities to a Practical Graph

To appreciate the distillation challenge, consider the data landscape we surveyed.

Source	**Entity Count**	**Domain**
PubChem	111,000,000	Chemical compounds
Icecat	15,000,000	Consumer products
ECHA C&L Inventory	5,000,000	Chemical classifications
Open Food Facts (DE)	800,000	Food products
Stiftung EAR	50,000	Electronics producers

SEPA Waste Thesaurus	10,000	Waste term synonyms
GESTIS	8,800	Lab safety substances
BSR Abfall-ABC	2,000	Waste item vocabulary
AVV Codes	840	Legal waste codes
Frankfurt Facilities	~50	Local disposal locations

Importing everything naively would produce a graph of over 100 million nodes, most of which would never be queried. A university chatbot that answers "How do I dispose of X?" needs a curated foundation, not a data lake. We estimated three practical scenarios for graph sizing.

Scenario	**Nodes**	**Use Case**
MVP	~25,000	Text-based disposal queries
With barcode scanning	~150,000	Product lookups via EAN codes
With full chemistry	~500,000	Lab waste advisor for university context

The MVP graph breaks down by layer: approximately 5,000 waste vocabulary items (primarily from the BSR Abfall-ABC and dictionary baseline), 8,000 synonym entries, 1,000 AVV codes, 10,000 disposal rules (the most labor-intensive layer, requiring extraction from municipal documents and statutes), 200 facilities, and a small number of container and building nodes. The product and chemistry layers would extend the graph on demand, via API connectors that resolve barcode queries against Open Food Facts or chemical name queries against PubChem at runtime, caching results as new graph nodes without requiring bulk import of millions of unused records.

Priority Matrix and Data Gaps

Based on the research, we organized the evaluated sources into three priority tiers.

- **Tier 1 (Must Have)**
    - The local Abfall-ABC as the authoritative waste vocabulary
    - Open Food Facts for food product packaging data
    - AVV codes from Gesetze-im-Internet for legal classification
    - PubChem and GESTIS for lab chemical hazard data
    - Wikidata or dictionary sources for multilingual synonyms and taxonomic relationships
- **Tier 2 (Should Have)**

    - ÖKOBAUDAT for construction material data
    - Icecat for electronics specifications
    - OpenStreetMap for Wertstoffhof locations
    - BAM dangerous goods database for ADR transport codes
- **Tier 3 (Nice to Have)**
    - UNSPSC and ECLASS for product taxonomy enrichment
    - Material property databases for density and safety data
    - Computer vision datasets for image recognition
    - iFixit for disassembly guidance

Data quality varied considerably across sources. Government databases such as ÖKOBAUDAT and the AVV are authoritative and well-structured but update infrequently. Crowdsourced platforms like Open Food Facts and Wikidata offer broad coverage and real-time updates but with inconsistent completeness: not every product has its packaging materials tagged, not every Wikidata item has German-language labels. Several important sources have no API at all: GESTIS prohibits scraping, LAGA guidelines are published as PDFs, and many municipal Sortierhilfen exist only as downloadable documents. This format heterogeneity (JSON APIs alongside PDF legal texts alongside SPARQL endpoints) directly motivated our decision to build two parallel ETL architectures.

The research also revealed notable gaps. No open database for everyday office and stationery items exists that is comparable to Open Food Facts for food. The EU Ecolabel ECAT database and Blauer Engel's product registry cover eco-certified products, but the majority of items a university procures (pens, printer cartridges, adhesives, cleaning agents) fall outside their scope. German university procurement catalogs, which would contain exactly these items, follow the BMEcat standard but are proprietary and accessible only through B2B platforms. A partnership with the university's procurement department would be the most practical path to closing this gap.

We deliberately excluded university-specific sources from this research phase, specifically campus bin locations, per-room container assignments, building-specific waste rules, and Schadstoffmobil schedules. Previous semesters had already attempted to collect this data through manual surveying and interviews with facilities management, with mixed results. We chose instead to focus on building foundational data layers that would benefit any deployment of the system, not just the Frankfurt campus. University-specific data can be layered on top of the foundation without schema changes, using the campus-scoped override mechanism described in the companion Data Engineering documentation.

Implications for the Data Strategy

The systematic research produced ~400 sources across ~20 topic areas, each documented with access methods, format specifications, licensing constraints, and relevance scores. This list is itself a deliverable, a reference that outlives this semester's implementation and provides a prioritized backlog for future work.

The discovered sources fall into two distinct categories that shaped our pipeline architecture. Structured sources (APIs, database dumps, CSV and JSON files with known schemas) can be imported through deterministic ETL pipelines: parse the input, map fields to graph properties, merge nodes and relationships. Open Food Facts, AVV code hierarchies, and facility databases fall into this category. Unstructured sources (PDFs, legal texts, municipal guidelines, German-language regulatory documents) require LLM-based extraction: analyze the document, identify entities and relationships, validate against the schema, and load the results with provenance tracking.

This duality directly motivated the two parallel pipeline architectures described in subsequent sections. Of the ~400 sources, we selected a few for implementation tests within the semester: the dictionary baseline for the vocabulary layer, Open Food Facts for the product layer, the AVV code hierarchy for the core layer, and the existing facility and waste item data from municipal sources. The remaining sources (chemical databases, municipal SaaS APIs, construction material registries, and the many PDF-based regulatory documents) represent a prioritized roadmap that can be executed incrementally using the pipeline infrastructure we built.

ETL and LLM Pipelines

The previous sections established what the knowledge graph should contain (for more details see the companion Data Engineering documentation), where the data would come from, and why a vocabulary foundation should be laid before anything else (also described in the companion document). What remains is the machinery to actually populate the graph based on the discovered information. As mentioned previously the sources can be split into two distinct categories: structured data (CSV files, JSON APIs, JSONL dumps, database exports) where the mapping from source fields to graph properties is deterministic, and unstructured data (PDFs of legal texts, municipal waste guides, German-language regulations) where a language model must be used to interpret prose and extract entities, relationships, or conditional logic.

Structured ETL

The structured pipeline (src/etl/) handles sources with known schemas through deterministic Python logic. Each script reads a specific data format, transforms it into the graph ontology, and writes to Neo4j using MERGE queries for idempotent re-runs. The facility import parses a JSON file of disposal locations into Facility nodes with addresses, opening hours, and contact information. The waste item import reads a CSV of the Abfall-ABC into WasteItem nodes with disposal relationships. The dictionary pipeline, the most complex structured import (described in the companion Data Engineering documentation), processes over a million JSON entries through concept clustering and batch import. Additional structured modules handle the AVV code hierarchy and the material taxonomy for product decomposition.

These pipelines share a common pattern: read the source, filter and transform in Python, write to Neo4j with MERGE-based idempotency (no LLM is involved). The code is fast (the entire dictionary imports in under a minute), deterministic (identical output on every run), and cheap (no GPU required and no API costs). For sources that arrive as structured data, this is always the preferred approach. The LLM pipeline exists for the sources where deterministic mapping is not possible.

Schema-Driven LLM Extraction

The central idea behind the LLM extraction pipeline is that the data model itself should drive the extraction. Rather than writing source-specific parsing logic for each document type, we defined Pydantic models, one for each entity type in the graph ontology, whose field names and descriptions serve simultaneously as the extraction schema and the LLM prompt. A WasteItemExtract model with a field disposal_targets: list[str] described as "Names of facilities or waste streams where this item should be disposed" tells the language model exactly what to look for, while Pydantic's validation ensures the output conforms to the expected structure.

The ExtractionChain is the workhorse component. It is stateless: given a block of text, a Pydantic schema, and optionally an extraction prompt and context string, it returns a list of validated entity instances. Internally, it wraps the schema in a list container (to handle documents containing multiple entities), constructs a prompt from the schema's field descriptions, and calls the language model with LangChain's structured output mechanism. If the model returns malformed output, the chain retries with the validation error as feedback.

The schema registry provides six extraction types that map to the graph ontology: WasteItemExtract for waste items with disposal targets and conditions, FacilityExtract for disposal locations with addresses and opening hours, WasteStreamExtract for bin types with physical descriptions, DisposalRuleExtract for conditional disposal logic (the "Paint Can" problem modeled as structured data), RelationshipExtract for connections between existing entities, and SourceExtract for provenance metadata. For domain-specific extraction tasks that do not fit these types (such as chemical compound identification or construction material classification), custom Pydantic models can be defined without modifying the pipeline code. A GenericEntityLoader handles any Pydantic model by creating labeled Neo4j nodes using the class name as the node label and serializing all fields as properties.

Two Modes: Chain and Agent

The Pipeline class offers two execution modes that trade speed for thoroughness.

Chain mode executes a single LLM call: text goes in, validated entities come out, and an optional database load persists the results with provenance tracking. This is fast (one API round-trip), deterministic at temperature zero, and suitable for well-understood document types where the extraction schema closely matches the source structure. A batch of Abfall-ABC entries, for instance, maps cleanly to WasteItemExtract without needing any additional context.

Agent mode interposes a multi-turn exploration phase before extraction. A LangGraph ReAct agent receives the input text and five read-only tools for inspecting the existing knowledge graph: search_entities lists entities of a given type, get_entity_details retrieves a specific node's properties, get_graph_stats returns node and relationship counts, list_waste_streams enumerates all bin types, and list_facilities enumerates all disposal locations. The agent uses these tools to build context about what already exists in the database, then passes this context, along with the original text, to the ExtractionChain for the actual extraction.

The practical difference is entity resolution accuracy. In chain mode, the pipeline might extract a facility name as "Wertstoffhof Frankfurt-Nord" when the graph already contains the same facility as "Wertstoffhof Nord," creating a near-duplicate that would require post-hoc deduplication. In agent mode, the agent discovers the existing name during its exploration phase and the extraction uses the canonical form. This graph-awareness comes at the cost of multiple LLM calls (several tool-use rounds before extraction begins), making agent mode significantly slower than chain mode. The choice depends on the source: high-volume structured imports lean more towards chain mode; while complex documents with many cross-references to existing entities should use agent mode.

Provenance and Idempotency

The graph schema introduced provenance tracking as a cross-cutting concern. The pipeline implementation makes this concrete through two mechanisms. First, deterministic UIDs: every entity receives a UID generated from the SHA256 hash of its canonical name, truncated to 16 characters. The same waste item extracted from different documents or at different times always resolves to the same Neo4j node, making re-runs safe, since the MERGE pattern updates existing nodes without creating duplicates. Second, the EntityLoader distinguishes between three actions: creation (new node), update (existing node with changed properties), and enrichment (existing node receiving additional properties from a new source), tracking each in a structured LoadResult that reports exactly what changed. When extraction quality improves through better models or refined prompts, the provenance trail identifies which entities came from which source, enabling targeted re-extraction without rebuilding the entire graph.

Models and Cost

The pipeline supports any OpenAI-compatible API endpoint, configured through environment variables (LLM_BASE_URL, LLM_API_KEY, LLM_MODEL). In practice, we used three model tiers depending on the task.

For reasoning-heavy extraction, involving documents containing conditional disposal logic, multi-step classification, or ambiguous German regulatory prose, we used the GPT-OSS reasoning model (in both its 20B and 120B variants). Its explicit chain-of-thought (when set to Medium or High) significantly improved extraction quality for complex disposal rules compared

to non-reasoning alternatives, particularly for identifying condition branches ("if the paint is still liquid" versus "if the can is empty and dry").

For high-volume batch classification (the dictionarys 92,000 concept clusters or the AVV code hierarchy), we tested Llama 3.1 8B via local API and Groq. Groq's inference speed in particular made it very practical to classify hundreds of thousands of entries at just $0.30 per 100,000 items. The tradeoff of this model however was the low reasoning capability, acceptable for binary classifications (is_physical, disposal_relevant) but insufficient for more nuanced extraction tasks.

GPT-4o-mini was also used during development and testing, providing a balance between capability and cost. The provider abstraction means that switching models requires changing a single environment variable, with no code modifications.

Extending the Pipeline

The pipeline was designed modular so that future semesters can add data sources without modifying the extraction framework itself. The process follows a repeatable pattern: define a Pydantic schema for the target entities (or reuse an existing one from the registry), write an extraction prompt describing what to look for, choose a mode (chain for structured or well-understood sources, agent for sources requiring graph context), run the pipeline against the source documents, and review the load results before committing to the database.

Prompt design follows a few practical guidelines we developed through iteration. Effective extraction prompts are specific about the target domain ("Extract waste items mentioned in this German municipal waste guide") rather than generic ("Extract entities"). They name the expected output format by referencing schema fields ("Include disposal_targets as facility or bin names"). And they constrain the language when relevant ("Preserve German terminology for waste item names; do not translate"). The dry_run option, available on both the Pipeline and the EntityLoader, executes the full extraction and validation pipeline without writing to Neo4j, allowing prompt iteration against real documents before any data enters the graph. In practice, two to three dry-run iterations were typically sufficient to refine an extraction prompt from producing noisy, partially correct output to producing clean, loadable entities. Subsequent sections will describe how this pattern was applied to specific data sources: the dictionary baseline (described in the companion Data Engineering documentation), Open Food Facts products, AVV code classification, and office item imports.

Data Imports

The research identified ~400 sources spanning millions of potential entities. The pipeline architecture described in the previous Section provided the machinery to process them. The dictionary baseline laid the vocabulary foundation. What remained was the actual work of importing data into the graph. Due to time and resource constraints, we could not assess (let

alone import) all sources. Instead, we focused on building a foundation that later semesters could extend: the dictionary baseline for the vocabulary layer, Open Food Facts for the product layer, the AVV code hierarchy for the legal classification layer, and a set of substitutes for the office items domain where our preferred data source proved unavailable. This section describes the practical experience of each import: the numbers, the challenges, and the decisions that shaped the final graph.

Dictionary Baseline

The dictionary import is described in detail in the companion Data Engineering documentation. In summary: we parsed 355,660 German nouns from the Kaikki.org Wiktionary extraction, clustered them into 329,231 concepts using OdeNet synsets and union-find synonym grouping, imported them as DictionaryEntry nodes with synonym, translation, and taxonomy edges, and classified them using Llama 3.1 8B. The result was 423,717 nodes, 589,124 edges, and 67,912 entries promoted to the WasteItem label, a 124-fold increase over the previous graph's 548 items.

Open Food Facts

Open Food Facts was the largest single import and the primary data source for the product layer. The dataset is available in two formats: a JSONL dump (2.6 GB, 719,174 products, a curated higher-completeness subset) and a Parquet file on Hugging Face (4.42 GB, 4,290,376 products, the complete database). We chose the Parquet format because it contains six times more products, supports columnar access through PyArrow without loading the entire file into memory, and includes image metadata and structured packaging data that the JSONL subset omits.

The import extracted 11 of the Parquet file's 111 columns. Product identity fields (barcode, name, brand, quantity) became Product node properties, with multilingual name resolution prioritizing German over English over the main language. Category tags, label tags, and country tags were stored as list properties for downstream filtering. The most valuable field for our use case was the structured packaging array: each element represents a physical packaging component (jar, lid, film, can) with its material, shape, recycling hint, unit count, and measured weight. These became PackagingComponent nodes linked to their parent Product via HAS_COMPONENT edges and to Material nodes via MADE_OF edges, enabling the full decomposition chain from barcode scan to per-component waste stream routing.

The import processed all 4.29 million products by streaming the Parquet file in batches of 50,000 rows and writing to Neo4j in transactions of 5,000 records across four passes: Product nodes first, then PackagingComponent nodes with HAS_COMPONENT edges, then MADE_OF edges to the pre-imported material taxonomy, and finally DERIVED_FROM provenance edges. The material taxonomy (~90 Material nodes mapping OFF's packaging material tags to waste streams) had to be imported separately before the main OFF import, since the MADE_OF edges match on existing Material nodes.

The result: 4,290,312 Product nodes, 555,227 PackagingComponent nodes, 459,850 MADE_OF edges, and 385,542 products tagged as available in Germany. Of the packaging components, 5,213 (1.1%) referenced material tags not present in our 90-material taxonomy. These were flagged for future extension rather than silently dropped. The 100 skipped Parquet columns (nutrition facts, ingredient lists, eco-scores, editorial metadata) remain available in the source file for future LLM-based enrichment.

Office Items and the BMEcat Gap

We already described the absence of an open office supply database as the most significant data gap. The university procures office items through framework contracts with B2B distributors who deliver product catalogs in the BMEcat XML format, a rich data source containing EAN barcodes, product descriptions, eCl@ss classification codes, manufacturer information, and links to Safety Data Sheets. However, BMEcat catalogs are confidential B2B exchange documents. No openly downloadable catalog could be found, since access usually required an institutional procurement account. A partnership with the university's procurement department would be the most practical path to obtaining this data, but was not feasible within the semester.

We therefore evaluated 10-20 public barcode and product databases as potential substitutes and found that none provides structured material composition data for non-food everyday products. UPCitemdb (686 million barcodes) and EAN-Search.org (1.07 billion) have enormous coverage but store only product names and categories, not what the product is made of or how to dispose of it. Open Products Facts (the non-food counterpart to Open Food Facts) has the right schema but is effectively a ghost town with only 40,000 sparsely populated entries.

Instead, we imported two partial substitutes that collectively cover a subset of the office items domain. The EU Ecolabel product database, accessible through its public API, yielded 16,333 Product nodes covering eco-certified categories relevant to office environments: tissue paper products, hard surface coverings, cleaning products, paints and varnishes, printed paper products, and furniture. Open Beauty Facts, the cosmetics sibling of Open Food Facts, contributed 40,033 Product nodes covering personal care items (soaps, shampoos, cleaning agents) that overlap with university facility supplies. Both imports follow the same pattern as the main product import: deterministic UIDs from EAN barcodes, MERGE-based idempotency, and DERIVED_FROM provenance tracking.

The combined 56,000 product nodes from these substitutes are cross-referenceable by EAN with all other product sources. A Nivea hand soap that appears in both Open Beauty Facts and a future BMEcat catalog would merge into a single enriched Product node rather than creating a duplicate. This was a deliberate design choice: build the infrastructure for product data even when the ideal source is unavailable, so that future imports slot in without migration.

AVV Code Hierarchy

We already described the Abfallverzeichnis-Verordnung as a three-level hierarchy of 20 chapters, approximately 113 subgroups, and roughly 840 specific waste codes. Our import source, a CSV prepared by the previous semester that combines the standard AVV with university-specific entries from the Frankfurt UAS campus, extends this to 22 chapters and 1,505 codes, adding items like soldering paste, gas cylinders, and binder clips that do not appear in the standard catalog.

The import proceeded in two stages. The first was a straightforward structured ETL: the extended CSV was parsed into 1,640 AVVCode nodes with HAS_PARENT relationships forming the three-level tree.

The second stage, classifying WasteItem nodes against the AVV hierarchy using LLMs, proved more instructive. The naive approach would be to send each waste item name along with all 1,505 AVV codes as context and ask the model to pick the best match. This attempt failed for two reasons. First, the full code list requires approximately 20,000 tokens of context, which was considerably slower than the previous classification and even exceeded Groq's rate limits after roughly 12 calls (250,000 tokens per minute). Second, the 8B llama model we used simply could not handle the task at that context size: it either returned empty results or malformed JSON responses.

The solution was hierarchical classification in three steps. Step one presents only the 22 chapter descriptions (~272 tokens) and asks the model to select the correct chapter. Step two presents only the subgroups within that chapter (~5–30 codes). Step three presents only the specific codes within that subgroup (~5–15 codes). Each step validates that the returned code exists in the filtered set; if any step fails, the item is skipped. This reduced the context per call from 20,000 tokens to roughly 300–500, making it feasible for the 8B model and increasing inference speed.

A first batch of 500 items yielded a 99.8% success rate (499 classified, 1 failure). The pipeline mechanics (hierarchical narrowing, validation at each step, resume support for interrupted runs) proved robust. Classification quality, however, was limited by the input data: the current WasteItem population consists primarily of dictionary-promoted entries (words like "Diamant" and "Verwandtschaft" that are technically physical objects but not meaningful waste items), which biased results toward catch-all codes. Quality will improve substantially when the pipeline is run against real waste items from curated sources rather than the dictionary baseline's broad vocabulary.

Import Summary

The following table summarizes the current state of the knowledge graph after all imports.

Import	Nodes	Key Edges
Dictionary baseline	423,717	589,124 (synonym, translation, taxonomy, provenance)
Open Food Facts	4,290,312 Products + 555,227 Components	5,305,389 (HAS_COMPONENT, MADE_OF, DERIVED_FROM)
EU Ecolabel	16,333 Products	16,333 DERIVED_FROM
Open Beauty Facts	40,033 Products	40,033 DERIVED_FROM
AVV codes	1,640 AVVCode nodes	2,265 HAS_PARENT
AVV classification		499 CLASSIFIED_AS (first batch)
Facilities	38 Facility nodes	
Waste items (Abfall-ABC)	548 WasteItem + 5 WasteStream	1,067 (DISPOSED_AT, DISPOSED_IN)
Materials taxonomy	~90 Material nodes	IS_A + DEFAULT_STREAM

The graph now contains over 5.3 million nodes and approximately 6 million edges. The vast majority of nodes are Product nodes from Open Food Facts; the vocabulary and disposal logic layers are comparatively small but carry disproportionate value for the chatbot's core function of answering "How do I dispose of X?" The product layer's value will be realized in a future phase when an agent pipeline connects products to waste items through their packaging components, enabling the barcode-to-disposal-route chain described in the schema.

Results

The semester began with 548 waste items, 38 facilities, and a chatbot that could not answer most queries. It ended with a knowledge graph of over 5 million nodes and ~6 million edges, populated by nine distinct import pipelines, served from a single workstation, and immediately available to the chatbot agent.

	Start of Semester	End of Semester
WasteItem nodes	548	67,912
Total nodes	~590	~5,327,000
Total edges	~1,100	~5,950,000
Product coverage	0	4,290,312 products,

		555,227 packaging components
Legal classification	0	1,640 AVV codes, 499 classified
Provenance-tracked sources	0	9 import pipelines

These numbers are the combined output of every pipeline described in this document and its companion: the dictionary baseline, Open Food Facts, EU Ecolabel, Open Beauty Facts, AVV codes, the Abfall-ABC, and a materials taxonomy. The graph grew from a sparse collection of municipal waste entries to a multi-layered knowledge base spanning vocabulary, products, legal classification, and disposal infrastructure. All pipelines are reproducible from open-access data sources using either local, self hosted or cloud inference APIs; no paid data sources or dedicated compute infrastructure is required.

Data Mining Perspective

The systematic research led to ~400 evaluated sources across ~20 topic areas, each documented with access methods, format specifications, licensing constraints, and relevance scores. This catalog is itself a semester deliverable, a prioritized reference that outlives this semester's implementation and provides a ready-made backlog for future work. The evaluation reframed the project's central challenge: the problem was never "we have no data" but rather "we have access to over 100 million potential entities and need a distillation strategy."

Beyond the list itself, three structural discoveries shaped the overall architecture. First, the SaaS oligopoly finding (that three platforms consolidate most German municipal waste management) reduced what appeared to be 400 custom integrations to three API endpoints. Second, conditional logic appeared independently in every domain we researched: AVV mirror entries, contaminated vs. clean construction materials, liquid vs. dry paint, battery-containing vs. battery-free electronics. This confirmed that the schema's Condition-based branching was not overengineered for edge cases but reflects a fundamental structure in waste classification law. Third, regional disposal variation clusters around a small number of genuinely ambiguous items rather than being arbitrary across municipalities, validating the schema's scope-based override mechanism.

LLM extraction pipeline

The pipeline architecture was designed around a specific bet: that schema-driven extraction (where Pydantic models serve simultaneously as validation schemas and LLM prompts) would make the framework source-agnostic. The results validate this. Adding a new data source requires defining a Pydantic schema and writing an extraction prompt; no pipeline code changes are needed. The dry_run option proved essential in practice: two to three iterations against real documents were typically sufficient to refine an extraction prompt from noisy output to clean, loadable entities, all without touching the database. The chain/agent mode distinction also held up as intended: chain mode handled high-volume batch imports efficiently, while agent mode's

graph-aware entity resolution prevented the near-duplicate problem on complex documents with many cross-references to existing entities.

AVV hierarchical classification

The AVV classification pipeline demonstrated both the potential and the practical challenges of LLM-based classification at scale. The naive approach (presenting all 1,505 AVV codes in a single context window) failed on two fronts: processing would simply take too long even when using Groq in addition to the local models, and the 8B llama could not reliably select from 20,000 tokens of context. The hierarchical three-step solution (chapter, then subgroup, then specific code) reduced the context to 300–500 tokens per call and achieved a 99.8% success rate (499 of 500 items classified). In addition to that, two Groq-specific workarounds were required: switching from function calling to JSON mode because Groq's validation layer rejected valid JSON that its own model generated, and adding the literal word "json" to the prompt because Groq requires it when JSON mode is enabled. These are the kind of practical integration lessons that documentation rarely captures but that significantly affect implementation timelines.

Classification quality versus input quality

The AVV classification results revealed an important distinction between pipeline reliability and output quality. The pipeline mechanics (hierarchical narrowing, validation at each step, resume support) proved robust at 99.8% success. But the current WasteItem population consists primarily of dictionary-promoted entries: words that passed the "is this a physical, disposable object?" filter but are not meaningful waste items in any practical sense. This biased classification results toward generic catch-all AVV codes. The pipeline itself is not the bottleneck; the input data is. The same pipeline run against curated waste items from the Abfall-ABC or BSR vocabulary would produce substantially more specific and useful classifications.

Gaps & Limitations

The graph is broad but unevenly deep. The product layer (4.29 million nodes) dwarfs the disposal logic layer, which still contains only the original 548 Abfall-ABC entries with actual disposal routes. The most significant external gap is the absence of an open office supply database (BMEcat catalogs would be ideal but are proprietary). Chemical substance databases and the full corpus of municipal PDFs have been assessed and cataloged but not yet imported. The pipeline infrastructure to process these sources exists; the remaining constraint is time, not tooling. The specific gaps and their recommended resolution paths are detailed in the following section.

Future Work

The pipeline infrastructure, graph schema, and data sourcing catalog produced this semester provide a foundation that subsequent project teams can build on without starting from scratch.

The schema supports all anticipated data categories without modification. The ETL and LLM extraction pipelines are source-agnostic and extensible. What remains is primarily data work: running existing pipelines against assessed sources, importing data that requires institutional access, and connecting the product layer to disposal knowledge. This section organizes the remaining work by effort and dependency.

Immediate Next Steps

The following tasks require no new infrastructure, no institutional partnerships, and no additional research. The pipelines exist and have been validated at proof-of-concept scale.

Completing the AVV classification

The hierarchical classification pipeline described in this document classified 500 WasteItem nodes against the AVV code hierarchy at a 99.8% success rate. The remaining ~67,000 WasteItem nodes can be processed with the same pipeline without modification. Classification quality will improve as the WasteItem population shifts from dictionary-promoted entries (which bias toward generic catch-all codes) to curated items from domain-specific sources.

Municipal vocabulary expansion

Berlin's BSR Open Data provides over 2,000 waste items with synonyms, avoidance tips, and recycling instructions under a CC BY license. Because the waste items themselves are universal across German cities, the BSR vocabulary can serve as a high-quality enrichment layer on top of the dictionary baseline, adding curated disposal knowledge to items that currently have only dictionary-level metadata. The import follows the same MERGE-based pattern as existing structured ETL pipelines.

Data Source Integration

The source list has over 400 sources, of which only a handful have been imported. The following categories represent what we think are the highest-priority remaining integrations.

Municipal SaaS APIs

The discovery that three platforms, Abfall+ (abfall.io), MyMüll (JumoMind), and Müllmax, consolidate most German municipal waste management is the single most actionable finding from the source research. Rather than building custom scrapers for each of Germany's 400+ districts, three API integrations would provide access to waste item vocabularies, sorting rules, collection schedules, and facility data for the majority of German university towns. The open-source Home Assistant integration (hacs_waste_collection_schedule) has already reverse-engineered these APIs and provides pre-configured tenant keys and municipality IDs for hundreds of cities, substantially reducing the integration effort. Imported municipal data would populate the disposal rules layer, currently the shallowest part of the graph, with city-specific sorting instructions and conditional logic.

Chemical substance databases

For a university with chemistry and biology laboratories, hazardous waste classification is a critical use case. GESTIS, the German occupational safety database maintained by the DGUV, provides 8,800 substance entries with storage classes, handling instructions, and regulatory references directly relevant to lab waste segregation. PubChem offers GHS hazard classifications for 111 million compounds, though the university context requires aggressive filtering to perhaps 500–2,000 common lab chemicals. The BAM dangerous goods database, which became freely accessible in July 2025, provides UN transport numbers and ADR classification codes for 4,500 substances. All three sources have been assessed and documented with access methods and format specifications.

LLM extraction from unstructured documents

The orchestrator pipeline, the four-stage LangGraph state machine for processing multi-entity documents, has been tested on individual documents but not yet run against the full corpus of sources identified during the research phase. Municipal waste guides, LAGA technical guidelines, university-specific chemical waste procedures, and AVV legal commentary documents remain as extraction targets. The pipeline infrastructure exists; the remaining work is prompt engineering for each document category and validation of extraction quality through dry-run iterations.

Institutional Partnerships

Two data gaps identified during the source evaluation cannot be closed through publicly available sources alone.

University procurement catalogs

The most significant data gap is the absence of an open database for everyday office and stationery items. German universities procure these items through framework contracts with B2B distributors who deliver product catalogs in the BMEcat XML format, a structured standard containing EAN barcodes, product descriptions, eCl@ss classification codes, manufacturer information, and links to Safety Data Sheets. These catalogs contain exactly the items the university purchases and needs to provide disposal guidance for. However, BMEcat catalogs are confidential B2B exchange documents, and access requires an institutional procurement account. A partnership with the university's procurement department is the most practical path to obtaining this data. A BMEcat parser would follow the same pattern as the existing structured ETL pipelines: parse XML, map fields to graph properties, MERGE into Neo4j with provenance tracking. The EAN-based deduplication already built into the product layer means that items appearing in both a BMEcat catalog and Open Food Facts would merge into a single enriched Product node rather than creating duplicates.

University-specific infrastructure data

Campus bin locations, per-room container assignments, building-specific waste rules, and Schadstoffmobil schedules are not published in any open dataset. Previous semesters attempted to collect this data through manual surveying and interviews with facilities management, with mixed results. The graph schema already supports this data through its infrastructure layer (Building → Room → Container nodes with WasteStream typing), and the disposal rule override mechanism (campus-scoped rules superseding city-level defaults) is designed for exactly this use case. The remaining challenge is establishing a reliable data collection process, ideally a collaboration with the university's facilities management department that provides periodic updates rather than a one-time manual survey.

Connecting Products to Disposal Knowledge

The product layer currently contains 4.29 million Product nodes and 555,227 PackagingComponent nodes with material links, but these products are not yet connected to the disposal knowledge in the core layer. The barcode scanner can decompose a product into its packaging components and route each component to a waste stream via the material taxonomy, but it cannot yet answer "How do I dispose of the product itself?" (as opposed to its packaging) without this connection.

Bridging this gap requires matching products to WasteItem categories: determining, for instance, that a jar of Nutella maps to the WasteItem "Schokoladencreme" (Biotonne) while its glass jar maps to "Altglas" (Altglascontainer) via the packaging decomposition chain. This matching can be approached in two ways: deterministic matching via Open Food Facts category tags (which map product categories like "Chocolates" to waste categories), or LLM-based matching where a language model infers the appropriate WasteItem from the product name and category. The deterministic approach is faster and cheaper; the LLM approach handles edge cases better. Both can run against the existing product and WasteItem nodes without schema changes.

Broader product database integration, incorporating UPCitemdb for non-food products and retail product data for electronics and household items, would extend the barcode scanner's coverage beyond food and cosmetics. The EAN-based deduplication mechanism ensures that products appearing in multiple sources merge correctly.

Graph Quality and Maintenance

As additional data sources are imported, two maintenance tasks will become necessary.

Material taxonomy extension

Of the 555,227 packaging components imported from Open Food Facts, 5,213 (1.1%) reference material tags are not present in the current 90-entry taxonomy. These were flagged during import rather than silently dropped. Extending the taxonomy to cover these tags and mapping

them to waste streams is a bounded task that improves the precision of the packaging decomposition chain.

Classification refinement

The AVV classification and dictionary baseline classification pipelines are both designed to be re-run as input data improves. The current classification results are biased by the dictionary-promoted WasteItem population, which contains many entries that are technically physical objects but not meaningful waste items. As curated waste items from the BSR vocabulary, municipal APIs, and domain-specific databases enter the graph, re-running the classification pipelines will produce more specific and useful results. The provenance tracking built into every pipeline ensures that re-classified entities can be traced back to their source, and the MERGE-based idempotency means re-runs are safe.

References

Bansal, T. (2025, December 8). Multimodal GraphRAG: Why vector search is no longer enough. Artificial Intelligence in Plain English. https://ai.plainenglish.io/multimodal-graphrag-why-vector-search-is-no-longer-enough-c9f550e9e2d4

Chen, J., Wu, H., Pang, J., Wang, Y., Zhang, D., & Sun, C. (2025). Tool learning with language models: A comprehensive survey of methods, pipelines, and benchmarks. Vicinagearth, 2, Article 16. https://doi.org/10.1007/s44336-025-00024-x

Curry, B. (2026, January 17). Building a knowledge graph: A comprehensive end-to-end guide using modern tools. Medium. https://medium.com/@brian-curry-research/building-a-knowledge-graph-a-comprehensive-end-to-end-guide-using-modern-tools-e06fe8f3b368

Moslemi, M. H., Mousavi, A., Behkamal, B., & Milani, M. (2026). Heterogeneity in entity matching: A survey and experimental analysis. Data & Knowledge Engineering, 164, 102575. https://doi.org/10.1016/j.datak.2026.102575

Ortego, D., Rodríguez, M., Almagro, M., Dahiya, K., Jiménez, D., & SanMiguel, J. C. (2026). Large language models meet extreme multi-label classification: Scaling and multi-modal framework. arXiv. arXiv:2511.13189

Silfverskiöld, I. (2025, December 21). How to build an over-engineered retrieval system. Data Science Collective. https://medium.com/data-science-collective/how-to-build-an-over-engineered-retrieval-system-5428444672d8

Tomar, R. K. (2026, January 13). 9 RAG architectures that save you months of dev work. Towards AI. https://pub.towardsai.net/9-rag-architectures-that-save-you-months-of-dev-work-0e219eb637cb

Wang, D. Y.-B., Shen, Z., Mishra, S. S., Xu, Z., Teng, Y., & Ding, H. (2025). SLOT: Structuring the output of large language models. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track (pp. 472–491). Association for Computational Linguistics.

World Wide Web Consortium. (2013). PROV-O: The PROV Ontology. W3C Recommendation.

Yadav, D. (2026, February 8). 5 LLM workflow patterns for building scalable AI applications: A complete guide (with examples). Towards AI. https://medium.com/@yadavdivy296/5-llm-workflow-patterns-for-building-scalable-ai-applications-a-complete-guide-376b7d7ccd1b